NYC Taxi Trips - Urban Mobility Data

Technical Report

A. Problem Framing and Dataset Analysis

Modern or present urban transportation systems generate large volumes of data that can reveal important mobility and economic patterns when properly processed. The objective of our system project is to design and implement a full-stack analytical system that transforms raw New York City Taxi and Limousine Commission (TLC) datasets into an interactive dashboard for exploring urban mobility behavior.

The system integrates three official TLC data sources:

- **Trips table(yellow_tripdata)** containing timestamps, distances, fares, payment methods, and pickup/dropoff location identifiers.
- **Zone Lookup Table(taxi_zone_lookup)** providing categorical mappings between location IDs, boroughs, and service zones.
- **Spatial Metadata(taxi_zones.geojson)** defining polygon boundaries for each taxi zone to support geographical visuals.

Our project system uses the above raw files in its system. But first they must be cleaned up, normalized before we use them in analysis.

Data Challenges

Under the process of inspecting the data files we encountered several quality issues while exploring files and under pipeline development:

- Missing or malformed timestamp fields.
- Logically invalid records (negative distances, negative fares, and negative trips durations, dropoff time earlier than pickup time).
- Duplicated records or near-duplicated trip entries.
- Extreme outliers in average speed and fare-per-distance ratios.

These challenges made it unsuitable to immediately insert them into the database without preprocessing.

Data Cleaning and Assumptions

Due to the challenges mentioned above we came with some decisions guided for preprocessing pipeline:

- All timestamps were standardized into a uniform datetime format.
- Numeric fields (distance, fare, duration) were explicitly cast and validated.
- All records with negative values or with zero distances or fare were excluded.

- Duplicated rows were removed.
- Outlier suspicious logs were logged rather than removed.

To keep track of the pipeline cleaning process, we ensured that all excluded or suspicious records were written to a separate file (excluded_records.csv) with the exclusion reason for later inspection and reporting.

Feature Engineering

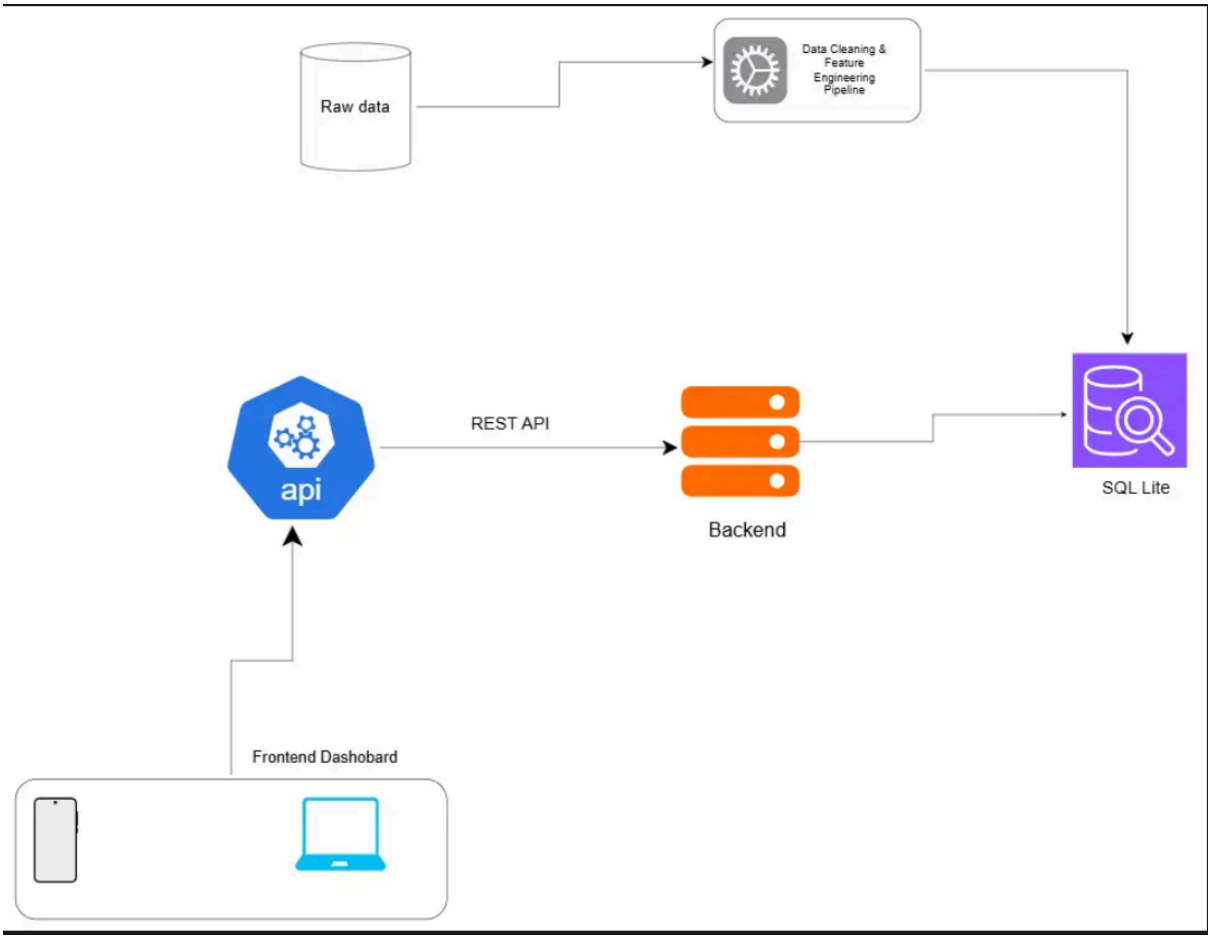
We implemented three derived features to provide deeper analytical insights:

1. **Trip Duration (minutes):** computed from pickup and dropoff timestamps.
2. **Average Speed (distance/duration):** highlights abnormal trips and traffic behavior.
3. **Revenue per Distance (fare/distance):** provides an economic efficiency indicator.

We improved and added additional temporal attributes (hour of day, day of week, peak-hour flag) to support time analysis. These features reduced query complexity at runtime and support more meaningful insights in dashboard generation.

B. System Architecture and Design Decisions

Architecture Overview



- **Frontend:** Static web dashboard built with HTML, CSS, and JavaScript.
- **Backend:** Flask REST API providing filtered queries, summaries, and geospatial data.
- **Database:** SQLite relational database storing cleaned and normalized trip records.

Zone metadata (CSV and GeoJSON) is served directly from cleaned static files to the frontend through backend routes, avoiding unnecessary joins for map rendering resulting in delay of providing visual display in frontend.

Design

- **Flask** was chosen for its simplicity and suitability for easy manipulation of whole endpoints.
- **SQLite** ensures easy reproducibility and eliminates external configurations requirements.
- **Static zone files** reduce databases overhead for large polygon geometries.
- **Split dashboard payloads (include_trips, include_summary)** improve performance by allowing the frontend to load tables and maps independently from aggregate computations.

C. Algorithmic Logic and Data Structures

The backend implements an iterative merge sort algorithm (manual_merge_sort_trips).
This algorithm is used when the API parameter custom_sort=true is enabled, allowing filtered trip results to be sorted without relying on Python's built-in sort() or database ordering.

The algorithm supports:

- Sorting large filtered trip datasets.
- Sorting by numeric and textual fields.
- Stable ordering for consistent frontend display.

Pseudo Code

```
function mergeSort(array):
  width = 1
  while width < length(array):
    for i from 0 to length(array) step 2*width:
      merge array[i:i+width] and array[i+width:i+2*width]
    width = width * 2
  return array
```

Complexity Analysis

- **Time complexity:** O(n log n)
- **Space complexity:** O(n) auxiliary buffer.

D. Insights and Interpretation

The urban mobility system enables analysis through SQL aggregation endpoints and interactive visualizations. We derived three insights as mentioned below

1. **Temporal Demand Peaks:** Hourly analysis reveals that taxi demand increases significantly during morning and evening commuting periods, with peak activity observed during typical rush hours.
   Method: Temporal aggregation using the temporal_analysis endpoint and hourly distribution charts.
2. **Spatial Concentration of Trips:** Pickup and dropoff heatmaps shows strong spatial clustering in central business districts and transportation hubs. Certain dominate trip volume, indicating major mobility corridors within the city.
   Method: trop_routes combined with map_based visualization.
3. **Economic Efficiency Variation:** Revenue-per-distance and average speed metrics vary significantly by time of the day and location. Late-night trips often show higher revenue per distance, reflecting reduced congestion and different fare patterns.
   Method: derived feature analysis using filtered summary statistics and charts.

E. System Performance and Reliability

Backend Optimizations

- In-memory response caching with time-to-live (TTL).
- Filter-based cache keying to prevent redundant aggregate computation.
- Split dashboard endpoints to avoid blocking frontend rendering.
- Indexed database schema for faster filtering and pagination.

Frontend Optimizations

- Debounced filter inputs to reduce API call frequency.
- AbortController to cancel stale requests.
- Short-lived client-side cache for repeated queries.

F. Reproducibility and Execution

To reproduce the system:

1. Ensure cleaned data files exist in **cleaned_data/**.
2. Run database initialization: **python backend/init_db.py**
3. Start backend server: **python backend/app.py**
4. Serve frontend: **python -m http.server 5500**
5. Open: **http://localhost:5500/nyc-frontend/index.html**

This project demonstrates system design from raw data to cleaning data under a pipeline, database integration, algorithmic processing, manual processing, and interactive user-friendly visualization.

The visuals tell a story about how cities move, how people travel and how data can reval meaningful patterns in complex urban systems.